SOFTWARE RELIABILITY

1. Introduction

Software systems are used in critical environments such as spacecraft, aircraft, nuclear power plants, and pacemakers. In these systems, any failure can be catastrophic. Therefore, reliability is a fundamental requirement.

1.1 Key Approaches to Dependability

Two main techniques are used to make software dependable:

- Fault Avoidance: The primary goal of good software engineering. The aim is to prevent faults from entering the system in the first place by following best practices during design and coding.
- Fault Tolerance: This technique accepts that some faults will exist and focuses on reducing their impact. It uses fault forecasting (predicting failures) and fault removal (testing and inspection).

1.2 Motivating Examples

The following real-world scenarios show why reliability analysis is important.

Example 1: Printer Reliability Test

A company tests a printer before purchase. The vendor claims one toner change is needed every 10,000 pages. The company runs a test and records failures at:

- 4,000 pages
- 6,000 pages
- 10,000 pages
- 11,000 pages
- 12,000 pages
- 15,000 pages

Analysis: 6 failures occurred in one toner cycle. The goal was at most 1 failure per cycle. Since the system is far below the target reliability, the company should NOT purchase this printer.

Example 2: Web Server Software

A web server program is developed with a failure intensity objective of 1 failure per 100,000 transactions.

During testing: 50 hours of testing, 10,000 transactions/hour average, 0 failures observed.

© Software Engineering Study Notes | For University Exam Preparation

Analysis: While 0 failures is promising, we need statistical confidence before release. Simply observing 0 failures does not guarantee the objective is met. Proper reliability models must be applied.

Example 3: Warranty Cost Calculation

A software product costs $50 per unit. The company offers one year of free updates. Failure intensity at release: $\lambda = 0.01$ failures/month.

Analysis: The expected number of failures per year must be calculated to estimate warranty service cost. This cost is then added to the unit price to determine the final selling price.

2. Software Reliability Concepts

2.1 Definition of Software Reliability

Definition: Software reliability is the probability that a software system will function properly without failure over a certain period of time.

It is one of the most important software quality attributes. It is an external quality attribute, which means it is observed from the outside (during operation), but it relates internally to the presence of faults or defects inside the code.

2.2 Fault vs. Failure

These two terms are often confused. Understanding the difference is critical for exams.

Term	Definition	Key Characteristic	Example
Fault (Bug)	A defect or error in the program code	Static — exists in the code	An off-by-one error in a loop condition
Failure	An occurrence where program behavior deviates from requirements during execution	Dynamic — happens at runtime	The program crashes or produces wrong output when the loop runs

Important Rule: A single fault can cause multiple failures. The failure only occurs when the program is executed under specific conditions that trigger the fault.

2.3 Sources of Software Faults

- Incorrect requirements: The specification itself is wrong, even if the code correctly implements it.
- Implementation errors: The design or code deviates from correct requirements.
- Unexpected operational changes: The system is used in ways not anticipated during development.
- Incorrect modifications: Updates or patches introduce new defects.

2.4 How Reliability Improves Software Engineering

Software reliability measures are used to:

- Quantitatively evaluate different software development technologies.
- Track the status of software development and testing.
- Plan and monitor upgrade and maintenance activities.
- Decide when to release software based on reliability targets.

© Software Engineering Study Notes | For University Exam Preparation

2.5 Hardware Reliability vs. Software Reliability

Characteristic	Hardware Reliability	Software Reliability
Stability over time	Tends to be stable or constant	Changes during test periods (reliability growth)
Source of faults	Physical wear and deterioration	Design issues (logical errors)
Nature of faults	Physical (corrosion, breakage)	Logical (wrong algorithm, wrong condition)
Repair approach	Replace or fix physical component	Patch or correct code
Failure pattern	Bathtub curve (early failures, stable, wear-out)	Decreasing failure rate if bugs are fixed

3. Execution Variables and Failure Behavior

3.1 Execution Exposure Variables

Reliability models express the probability of failure over a certain exposure variable. This variable measures how much the system has been used. The most common options are:

Variable	Description	Used By
Execution Time (CPU Time)	The actual time the processor spends executing program instructions	Engineers, researchers
Clock Time (Elapsed Time)	Time from start to end of program execution (includes idle time)	Developers during testing
Calendar Time	Regular everyday time (hours, days, months)	Managers and users

Key Rule: Reliability is usually computed using execution time, then converted to calendar time for management reporting. Clock time equals execution time only when CPU utilization is constant.

Other exposure variables include: number of executed test cases, number of program runs, and number of transactions.

3.2 Failure Behavior

Failure occurrences are modeled as random variables. This is because:

- Programmers make mistakes in unpredictable ways.
- The conditions under which programs execute are unpredictable.

Using time as the variable, failures can be expressed in four ways:

1. Time of failure (exact moment the failure occurs)
2. Time interval between consecutive failures
3. Cumulative failures occurred up to a specified time
4. Number of failures occurring in a specific time interval

3.3 Mean Value Function and Failure Intensity Function

Two important mathematical functions describe failure behavior:

Mean Value Function — m(t)

This function gives the expected (average) number of failures that have occurred from the start up to time t.

Formula: m(t) = E[M(t)]

Where M(t) is the random variable representing failure count at time t, and E[·] is the expected value.

Failure Intensity Function — λ(t)

This measures the rate of change of the mean value function — i.e., how many failures occur per unit of time.

Formula: $\lambda(t) = dm(t) / dt$

It is the derivative of the mean value function with respect to time. It tells us how rapidly failures are occurring at any given moment.

4. Reliability Metrics

Reliability metrics allow us to quantify how reliable a system is. They are derived from failure data collected during testing or operation. The four main metrics are:

Metric	Full Name	What It Measures	Best Used When
POFOD	Probability of Failure on Demand	Chance that a single service request will fail	Service requests are unpredictable or infrequent
ROCOF	Rate of Occurrence of Failures	How many failures occur per unit time (= failure intensity)	Services are requested regularly and continuously
MTTF	Mean Time to Failure	Average time between consecutive failures	Long transactions where continuity of service matters
Availability	Availability	Probability that the system is working at any given moment	Continuous service delivery is critical

4.1 POFOD — Probability of Failure on Demand

Definition: POFOD is the probability that a system will fail when a service request is made.

Simple Explanation: If you press a button, POFOD tells you the chance it will fail to work.

Example: POFOD = 0.001 means 1 failure in every 1,000 requests.

Use Case: Best for systems where requests are unpredictable — e.g., a safety alarm that is tested randomly.

4.2 ROCOF — Rate of Occurrence of Failures

Definition: ROCOF is the number of failures occurring per unit time. It is equivalent to the failure intensity function $\lambda(t)$.

Simple Explanation: It measures how often the system fails in a given time window.

Example: ROCOF = 2 failures/hour means the system fails twice every hour on average.

Use Case: Best for systems where users access the service frequently and regularly — e.g., ATM networks or online booking systems.

4.3 MTTF — Mean Time to Failure

Definition: MTTF is the average time between consecutive system failures.

Formula: $MTTF = 1 / \lambda$ (where λ is the failure rate)

Simple Explanation: If $\lambda = 0.001$ failures/hour, then MTTF = 1/0.001 = 1,000 hours.

Example: A database server with MTTF = 500 hours fails once every 500 hours on average.
Use Case: Best for long-running transactions where service continuity is critical — e.g., banking systems or flight control systems.

4.4 Availability

Definition: Availability is the probability that the system is operational (working correctly) at any given instant in time.
Formula: Availability = Uptime / (Uptime + Downtime)
Simple Explanation: A system that is up 99 hours out of 100 has 99% availability.
Use Case: Best for systems where continuous service is expected — e.g., hospitals, e-commerce websites, telecommunications.

The following table shows standard availability classes and their acceptable downtime:

Class	Availability (%)	Max Downtime (min/year)	System Type
1	90.000%	52,560	Unmanaged
2	99.000%	5,256	Managed
3	99.900%	526	Well-managed
4	99.990%	52.6	Fault-tolerant
5	99.999%	5.3	Highly available
6	99.9999%	0.53	Very highly available
7	99.99999%	0.0053	Ultra available

5. Reliability Models

A reliability model mathematically describes how failures occur over time. It uses a stochastic (random) process to capture the uncertain nature of software failures.

Key assumptions:

- Failures are independent of each other — one failure does not directly cause the next.
- The state of the software changes as failures are detected and repaired.

5.1 Three Basic Reliability Models

Model	What It Describes	Purpose
Usage Model	How the software is used by users (input patterns, frequency)	Captures real-world operational environment
Trend Model	How reliability changes over time as bugs are fixed or new bugs are introduced	Tracks reliability growth or degradation
Probabilistic Failure Model	The random nature of when failures occur	Mathematically models failure randomness

5.2 Failure Probability and Reliability Formulas

Failure Probability F(t)

This is the probability that a failure occurs at or before time t.

Formula: $F(t) = P(T \leq t) = \int_0^t f(u)\, du$

Variables: T = time of failure (random variable), f(u) = failure density function (probability distribution of failure time)

Reliability R(t)

This is the probability that the system works correctly throughout the entire time interval [0, t].

Formula: $R(t) = 1 - F(t) = P(T \geq t) = \int_t^{\infty} f(u)\, du$

Meaning: Reliability is the complement of failure probability. If F(t) = 0.05 (5% chance of failure by time t), then R(t) = 0.95 (95% chance of working until time t).

Failure Density f(t)

Formula: $f(t) = dF(t)/dt$

Meaning: The probability distribution function of the failure time T. It describes how likely a failure is at any given instant.

6. Deferred Repair and Reliability Growth

6.1 Deferred Repair

Definition: Deferred repair is the assumption that faults are NOT fixed immediately when a failure is discovered. Repair is postponed until the next software release.

When it applies: During operation (after release to customers). Fixes are batched and released together in the next version.

Effect on failure intensity: The failure intensity λ remains CONSTANT because no faults are being removed from the current version.

Real-world example: A banking application released to customers. Known bugs are logged, but only fixed in the next quarterly release. Meanwhile, the system continues operating with those bugs.

6.2 Reliability Growth

Definition: Reliability growth is the phenomenon where reliability improves over time because failures are detected and the corresponding faults are fixed.

When it applies: During testing and production, when faults are actively found and removed.

Effect on failure intensity: The failure intensity λ DECREASES as faults are removed.

Real-world example: During beta testing, engineers find and fix bugs regularly. As the test progresses, failures become less frequent — reliability grows.

6.3 Exponential Reliability Model (Under Deferred Repair)

Under the deferred repair assumption, where λ is constant, the reliability model is derived as follows:

Derivation Logic:

1. Divide the time interval [0, t] into i equal segments, each of length t/i.
2. The probability of failure in each segment is approximately λ·(t/i).
3. Reliability R(t) = probability of no failure in any segment:
   $R(t) = (1 - \lambda t/i)^i$
4. As i → ∞ (infinitely small segments), this converges to:
   $R(t) = e^{(-\lambda t)}$

This is the Exponential Reliability Model.

Variables:

- R(t) = Reliability at time t (probability of no failure by time t)
- λ = Failure rate (failures per unit time), constant under deferred repair

- t = Observation time
- e = Euler's number ≈ 2.71828

Worked Example

Given: λ = 0.001 failures/hour, t = 8 hours
Find: R(t) = ?

Step 1: R(t) = e^(-λt)
Step 2: R(8) = e^(-0.001 × 8)
Step 3: R(8) = e^(-0.008)
Step 4: R(8) ≈ 0.992

Answer: There is approximately a 99.2% probability that the system will work without failure for 8 hours.
Interpretation: For λ = 0.001 (1 failure per 1,000 hours), the system is highly reliable for a short 8-hour period.

7. Availability, MTTF, MTTR, and MTBF

While reliability measures the probability of working over a time interval, availability measures the probability of working at any specific instant — including after repairs.

7.1 Key Metrics Defined

Metric	Full Name	Definition	Formula
MTTF	Mean Time to Failure	Average time from system start (or restart) until the next failure	MTTF = 1/λ
MTTR	Mean Time to Repair	Average time taken to restore the system after a failure	MTTR = 1/μ
MTBF	Mean Time Between Failures	Average time between the start of one failure and the start of the next	MTBF = MTTF + MTTR

7.2 Relationship Between Metrics

Key Relationship:
MTBF = MTTF + MTTR

How it is derived:
Let:

- A = probability the system is UP (available)
- U = probability the system is DOWN (unavailable)
- λ = failure rate = 1/MTTF (rate at which system goes from UP to DOWN)
- μ = repair rate = 1/MTTR (rate at which system goes from DOWN to UP)

Since A + U = 1, and in steady state: A × λ = U × μ (rate of going down = rate of coming up):
Solving: A = μ/(λ + μ) = MTTF/(MTTF + MTTR) = MTTF/MTBF
U = λ/(λ + μ) = MTTR/(MTTF + MTTR) = MTTR/MTBF

Approximation: Since MTTF >> MTTR in most systems, unavailability can be approximated as:
U ≅ MTTR/MTTF

7.3 Availability Formula Summary

Availability Formula 1 (Direct):
A = Uptime / (Uptime + Downtime)

Availability Formula 2 (Using Rates):
A = μ / (λ + μ) where λ = 1/MTTF and μ = 1/MTTR

Availability Formula 3 (Using MTTF/MTBF):
A = MTTF / MTBF = MTTF / (MTTF + MTTR)

7.4 Worked Example — Availability Calculation

Problem: A product must be available 99% of the time and downtime is 6 minutes per failure. Find λ (failure rate) and MTTF.

Given:

- Availability A = 99% = 0.99
- MTTR = 6 minutes
- Unavailability U = 1 - A = 0.01

Step 1: Use the approximation U ≅ MTTR/MTTF
0.01 = 6 / MTTF

Step 2: Solve for MTTF
MTTF = 6 / 0.01 = 600 minutes

Step 3: Calculate failure rate λ = 1/MTTF
λ = 1/600 ≈ 0.00167 failures/minute = 0.1 failures/hour

Answer: MTTF ≈ 594–600 minutes (approximately 10 hours). Failure rate λ ≈ 0.1 failures/hour.
Note: The textbook gives MTTF = 594 min using a slightly more precise calculation.

8. Key Comparison Tables

8.1 Reliability vs. Availability

Aspect	Reliability	Availability
Definition	Probability of working correctly over a continuous time interval	Probability of working correctly at any given instant
Time focus	Time period [0, t]	Point in time
Formula	R(t) = e^(-λt)	A = MTTF / (MTTF + MTTR)
Considers repair?	No — assumes no repair during interval	Yes — includes repair/recovery time
When to use	Long-running operations without interruption	Systems that can be repaired and restarted

8.2 MTTF vs. MTTR vs. MTBF

Metric	Meaning	Formula	Practical Meaning
MTTF	Mean Time to Failure	1/λ	How long the system works before failing
MTTR	Mean Time to Repair	1/μ	How long it takes to fix the system after failure
MTBF	Mean Time Between Failures	MTTF + MTTR	Total cycle time from start of one failure to start of next

8.3 POFOD vs. ROCOF vs. MTTF vs. Availability

Metric	Measures	Use Case	Example
POFOD	Probability of failure per request	Unpredictable or infrequent requests	Safety systems, emergency alarms
ROCOF	Failures per unit time	Regular, continuous service	ATM networks, ticket booking
MTTF	Average time between failures	Long transactions, continuous operation	Flight control, banking servers
Availability	Fraction of time system is operational	Continuous service, repairable systems	Websites, hospitals, telecom

8.4 Deferred Repair vs. Reliability Growth

Aspect	Deferred Repair	Reliability Growth

Fault fixing	Faults are NOT fixed immediately	Faults ARE fixed as found
Failure intensity λ	Remains CONSTANT	DECREASES over time
When it occurs	During operation (production)	During testing and development
Reliability model	Exponential: R(t) = e^(-λt)	More complex — decreasing λ over time
Example	Software in use between releases	Beta testing with active bug fixes

9. Quick Revision Notes

9.1 Essential Definitions

- Software Reliability: Probability that software functions without failure over a time period.
- Failure: Deviation of program behavior from requirements during execution (dynamic).
- Fault (Bug): A defect in the code that triggers failure under specific conditions (static).
- Reliability Growth: Decrease in failure intensity over time as faults are removed.
- Deferred Repair: Assumption that faults are not repaired immediately; λ stays constant.
- Availability: Probability the system is operational at any given instant.
- MTTF: Average time a system works before failing (= 1/λ).
- MTTR: Average time to repair a system (= 1/μ).
- MTBF: Average time between start of consecutive failures (= MTTF + MTTR).

9.2 Essential Formulas

Exponential Reliability: $R(t) = e^{(-\lambda t)}$
Failure Probability: $F(t) = 1 - R(t) = 1 - e^{(-\lambda t)}$
MTTF: $MTTF = 1/\lambda$
MTTR: $MTTR = 1/\mu$
MTBF: $MTBF = MTTF + MTTR$
Availability: $A = MTTF / MTBF = \mu/(\lambda+\mu)$
Unavailability: $U = MTTR / MTBF = \lambda/(\lambda+\mu)$
Approx Unavailability: $U \cong MTTR / MTTF$ (when MTTF >> MTTR)
Failure Intensity: $\lambda(t) = dm(t)/dt$
Mean Value Function: $m(t) = E[M(t)]$

10. Exam Preparation

10.1 Frequently Asked Theory Questions

5. Define software reliability. How does it differ from hardware reliability?
6. What is the difference between a fault and a failure? Give an example.
7. What are the three basic software reliability models? Briefly explain each.
8. What is reliability growth? When does it occur?
9. What is deferred repair? How does it affect the failure intensity?
10. Define MTTF, MTTR, and MTBF. State the relationship between them.
11. What are the four main reliability metrics? When is each metric most appropriate?
12. What is the difference between reliability and availability?
13. Explain the exponential reliability model. What assumptions does it make?
14. What are execution variables? Name and explain the three types of time variables.

10.2 Frequently Asked Numerical Questions

15. Given λ, calculate R(t) using the exponential model R(t) = e^(-λt).
16. Given A and MTTR, find MTTF and failure rate λ.
17. Calculate MTBF from MTTF and MTTR.
18. Find availability A from MTTF and MTBF.
19. Compute unavailability U = 1 - A.
20. Determine warranty cost per unit given failure intensity and service cost per repair.
21. Given failure data (times/pages), determine if system meets reliability target.

10.3 Viva Questions and Answers

Q: What is the difference between fault and failure?

A: A fault is a static defect in the code (a bug). A failure is a dynamic event during execution where the program behaves incorrectly. A fault may cause many failures depending on conditions.

Q: Why is software reliability modeled as a stochastic process?

A: Because both the introduction of faults by programmers and the conditions under which software is executed are unpredictable and random in nature.

Q: What does reliability growth mean?

A: Reliability growth is the improvement in reliability over time as bugs are discovered and fixed during testing. The failure intensity decreases as faults are removed.

© Software Engineering Study Notes | For University Exam Preparation

Q: When is deferred repair assumed?

A: During operation, when the system is deployed and faults are not fixed immediately. Fixes are deferred until the next release, so λ remains constant.

Q: What is the formula for availability and what does each term mean?

A: A = MTTF / (MTTF + MTTR). MTTF is the average working time before failure. MTTR is the average repair time. Together they form MTBF, the total cycle time.

Q: If MTTF >> MTTR, how is unavailability approximated?

A: U ≅ MTTR / MTTF. This simplification works well when the system is much more reliable than the time taken to repair it.

11. One-Page Exam Revision Sheet

Use this sheet for final revision in the last 5–10 minutes before your exam.

DEFINITIONS

- Reliability: Probability of correct operation over a time interval
- Failure: Runtime deviation from requirements (dynamic)
- Fault: Code defect that causes failure under specific conditions (static)
- Reliability Growth: Decreasing λ as faults are removed during testing
- Deferred Repair: Faults not fixed immediately → λ stays constant
- Availability: Probability of being operational at any given instant

FORMULAS

- $R(t) = e^{(-\lambda t)}$ [Reliability — exponential model]
- $F(t) = 1 - R(t)$ [Failure probability]
- $\lambda(t) = dm(t)/dt$ [Failure intensity]
- $MTTF = 1/\lambda$ [Mean time to failure]
- $MTTR = 1/\mu$ [Mean time to repair]
- $MTBF = MTTF + MTTR$ [Mean time between failures]
- $A = MTTF / MTBF = \mu/(\lambda+\mu)$ [Availability]
- $U = MTTR / MTBF = \lambda/(\lambda+\mu)$ [Unavailability]
- $U \cong MTTR/MTTF$ (when MTTF >> MTTR)

METRICS QUICK GUIDE

- POFOD → Unpredictable/rare requests (e.g., safety systems)
- ROCOF → Regular continuous service (e.g., ATM, booking)
- MTTF → Long transactions, continuity needed (e.g., banking)
- Availability → Continuous service, repairable (e.g., websites)

RELIABILITY MODEL SUMMARY

- Usage Model: How users use the software
- Trend Model: How reliability changes over time
- Probabilistic Failure Model: Random nature of failure occurrences

TIME VARIABLES: Execution Time (CPU) | Clock Time (Elapsed) | Calendar Time (Daily)
KEY RULE: Compute with execution time → convert to calendar time for reporting

AVAILABILITY CLASSES (memorize key ones):

- Class 2: 99% available → 5,256 min/year downtime
- Class 4: 99.99% available → 52.6 min/year downtime

© Software Engineering Study Notes | For University Exam Preparation

- Class 5: 99.999% available → 5.3 min/year downtime (Highly available)
- Class 7: 99.99999% available → 0.0053 min/year (Ultra available)